



ESCOLA POLITÉCNICA DA UNIVERSIDADE DE  
SÃO PAULO

## Aula 07 – Métodos Espectrais

Resolução da tarefa

Renan de Luca Avila  
6 de novembro de 2025

# Sumário

|          |                                                                                             |           |
|----------|---------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Resumo</b>                                                                               | <b>2</b>  |
| <b>2</b> | <b>Exercício 1 — Integrais numéricas por quadraturas espectrais de Legendre e Chebyshev</b> | <b>2</b>  |
| 2.1      | Enunciado . . . . .                                                                         | 2         |
| 2.2      | Planejamento e fundamentos teóricos . . . . .                                               | 2         |
| 2.3      | Etapas de implementação e funções principais . . . . .                                      | 4         |
| 2.4      | Resultados e discussão . . . . .                                                            | 6         |
| 2.5      | Conclusão . . . . .                                                                         | 9         |
| <b>3</b> | <b>Exercício 2 — Transformações entre coeficientes e valores nodais (Legendre)</b>          | <b>10</b> |
| 3.1      | Enunciado . . . . .                                                                         | 10        |
| 3.2      | Planejamento e fundamentos teóricos . . . . .                                               | 10        |
| 3.3      | Etapas de implementação e funções principais . . . . .                                      | 10        |
| 3.4      | Resultados e discussão . . . . .                                                            | 11        |
| 3.5      | Conclusão . . . . .                                                                         | 13        |
| <b>4</b> | <b>Tarefas Extras – Ortogonalidade Analítica e Quadraturas de Chebyshev</b>                 | <b>13</b> |
| 4.1      | Enunciado . . . . .                                                                         | 13        |
| 4.2      | Planejamento e fundamentos teóricos . . . . .                                               | 14        |
| 4.3      | Etapas de implementação e funções principais . . . . .                                      | 15        |
| 4.4      | Resultados e discussão . . . . .                                                            | 16        |
| 4.5      | Conclusão . . . . .                                                                         | 18        |
| 4.6      | Observações adicionais . . . . .                                                            | 18        |
| <b>5</b> | <b>Códigos Python e Setup de Ambiente</b>                                                   | <b>19</b> |
| 5.1      | Configuração do ambiente Python . . . . .                                                   | 19        |
| 5.2      | Códigos Python na íntegra . . . . .                                                         | 20        |
| 5.2.1    | Exercício 1 – Quadraturas espectrais . . . . .                                              | 20        |
| 5.2.2    | Exercício 2 – Matriz de transformação de Legendre . . . . .                                 | 23        |
| 5.2.3    | Tarefas Extras – Ortogonalidade e Quadraturas de Chebyshev . . . . .                        | 24        |
| 5.3      | Observações finais . . . . .                                                                | 26        |
| <b>6</b> | <b>Declaração de uso de LLM</b>                                                             | <b>27</b> |

# 1 Resumo

Este relatório implementa, em Python, os exercícios apresentados nos slides da Aula 07 (Prof. Osvaldo Guimarães), e discute as soluções. O documento contém códigos completos, figuras e tabelas, além de apêndices com setup e fontes.

## 2 Exercício 1 — Integrais numéricas por quadraturas espectrais de Legendre e Chebyshev

### 2.1 Enunciado

Nesta tarefa, foi solicitado o cálculo numérico de quatro integrais reais em intervalos finitos, utilizando métodos espectrais de quadratura de alta precisão. O objetivo é comparar a eficiência e a taxa de convergência das regras de Gauss–Legendre, Gauss–Lobatto, Gauss–Radau e duas variantes baseadas em polinômios de Chebyshev (Fejér I e Clenshaw–Curtis).

As integrais propostas são:

$$\begin{aligned} I_1 &= \int_0^4 t e^{2t} dt, \\ I_2 &= \int_{-1}^0 \frac{\sin x}{x} dx, \\ I_3 &= \int_0^2 e^{x^2} dx, \\ I_4 &= \int_0^1 \frac{e^{x-1} - 1}{x - 1} dx. \end{aligned}$$

A precisão desejada é de erro relativo inferior a  $10^{-8}$ , sendo cada integral avaliada com  $n = 4, 8, 16, 32$  pontos para cada método. Os resultados numéricos devem ser comparados com valores de referência analíticos (quando disponíveis) ou obtidos por integração de alta precisão via `mpmath.quad`.

### 2.2 Planejamento e fundamentos teóricos

A integração numérica de uma função  $f(x)$  em um intervalo  $[a, b]$  é aproximada por:

$$\int_a^b f(x) dx \approx \sum_{i=1}^n w_i f(x_i),$$

onde  $\{x_i\}$  são os *nós* e  $\{w_i\}$  os *pesos* da quadratura.

**Fundamentos (Legendre).** As regras de Gauss–Legendre, –Lobatto e –Radau usam propriedades de ortogonalidade de  $P_n(x)$ :

$$\int_{-1}^1 P_m(x) P_n(x) dx = \frac{2}{2n+1} \delta_{mn}.$$

Fórmulas clássicas:

$$\text{Gauss: } w_i = \frac{2}{(1 - x_i^2) [P'_n(x_i)]^2}, \quad \text{Lobatto: } w_i = \frac{2}{n(n-1) [P_{n-1}(x_i)]^2},$$

$$\text{Radau (nó em } -1\text{): } w_i = \frac{1 - x_i}{n^2 [P_{n-1}(x_i)]^2}.$$

Para aplicar em  $[a, b]$ , mapeamos  $z \in [-1, 1] \mapsto x = \frac{b-a}{2}z + \frac{b+a}{2}$  e  $w = \frac{b-a}{2}w_z$ .

**Fundamentos (Chebyshev).** Fejér I e Clenshaw–Curtis usam  $T_k(x) = \cos(k \arccos x)$  com nós

$$x_k = \cos\left(\frac{(2k-1)\pi}{2n}\right) \quad (\text{Fejér I}), \quad x_k = \cos\left(\frac{k\pi}{n}\right) \quad (\text{Clenshaw–Curtis}),$$

e pesos obtidos por somas de cossenos (séries de Fourier discretas).

**Soluções analíticas de referência.**

(a)  $I_1 = \int_0^4 te^{2t} dt$ . Integração por partes com  $u = t$ ,  $dv = e^{2t} dt \Rightarrow du = dt$ ,  $v = \frac{1}{2}e^{2t}$ :

$$\int te^{2t} dt = \frac{t}{2}e^{2t} - \int \frac{1}{2}e^{2t} dt = \frac{t}{2}e^{2t} - \frac{1}{4}e^{2t} + C.$$

Logo

$$I_1 = \left[ \frac{t}{2}e^{2t} - \frac{1}{4}e^{2t} \right]_0^4 = \frac{7}{4}e^8 - \left( -\frac{1}{4} \right) = \frac{1}{4}(7e^8 + 1).$$

(b)  $I_2 = \int_{-1}^0 \frac{\sin x}{x} dx$ . Note que  $\frac{\sin x}{x}$  é par, então

$$I_2 = \int_{-1}^0 \frac{\sin x}{x} dx = \int_0^1 \frac{\sin x}{x} dx = \text{Si}(1),$$

onde  $\text{Si}(x) = \int_0^x \frac{\sin t}{t} dt$  é a integral do seno.

(c)  $I_3 = \int_0^2 e^{x^2} dx$ . Não há primitiva elementar, mas

$$\int e^{x^2} dx = \frac{\sqrt{\pi}}{2} \text{erfi}(x), \quad \text{erfi}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{t^2} dt.$$

Assim,

$$I_3 = \frac{\sqrt{\pi}}{2} \text{erfi}(2).$$

(d)  $I_4 = \int_0^1 \frac{e^{x-1} - 1}{x-1} dx$ . Com  $u = x - 1$  (de  $-1$  a  $0$ ):

$$I_4 = \int_{-1}^0 \frac{e^u - 1}{u} du = \sum_{k=1}^{\infty} \frac{1}{k!} \int_{-1}^0 u^{k-1} du = \sum_{k=1}^{\infty} \frac{1}{k!} \left[ \frac{u^k}{k} \right]_{-1}^0 = \sum_{k=1}^{\infty} \frac{(-1)^{k+1}}{k k!}.$$

Usando a expansão de  $\text{Ei}(x)$ :

$$\text{Ei}(x) = \gamma + \ln|x| + \sum_{k=1}^{\infty} \frac{x^k}{k k!},$$

obtemos, para  $x = -1$ ,

$$\sum_{k=1}^{\infty} \frac{(-1)^{k+1}}{k k!} = \gamma - \text{Ei}(-1).$$

Portanto,

$$I_4 = \gamma - \text{Ei}(-1),$$

onde  $\gamma$  é a constante de Euler–Mascheroni.

## 2.3 Etapas de implementação e funções principais

A implementação em Python 3.11 segue fielmente as fórmulas acima. Abaixo, destacamos trechos essenciais (o **código completo** está no apêndice do relatório, arquivo `/code/aula07_tarefa1_quadratras.py`).

**Mapeamento  $[-1, 1] \rightarrow [a, b]$  e integração genérica.**

```
def map_to_interval(z, w, a, b):
    x = 0.5*((b-a)*z + (b+a))
    ww = 0.5*(b-a)*w
    return x, ww

def integrate_rule(f, a, b, rule, n):
    z, w = nodes_weights(rule, n) # despacha p/ regra escolhida
    x, ww = map_to_interval(z, w, a, b)
    return float(np.sum(ww * f(x)))
```

*Comentário.* Implementa o mapeamento linear e a soma  $\sum_i w_i f(x_i)$ .

**Gauss–Lobatto:**  $x = \{-1, \text{raiz de } P'_{n-1}, 1\}$ ;  $w = \frac{2}{n(n-1)P'_{n-1}(x)^2}$ .

```
def gauss_lobatto(n):
    z = np.empty(n); z[0], z[-1] = -1.0, 1.0
    if n > 2:
        P = np.polynomial.legendre.Legendre([0]*(n-1) + [1.0])
        roots = np.sort(P.deriv().roots())
        z[1:-1] = roots
    Pn1 = np.polynomial.legendre.Legendre([0]*(n-1) + [1.0])(z)
    w = 2.0 / ((n-1)*n*(Pn1**2))
    return z, w
```

*Comentário.* Usa as raízes de  $P'_{n-1}$  e aplica a fórmula de pesos clássica.

**Gauss–Radau (nó fixo em  $-1$ ) via matriz tridiagonal (Jacobi).**

```
def gauss_radau_left_tridiag(n):
    j = np.arange(0, n-1)
    alpha = 1.0 / ((2*j+1)*(2*j+3))
    j2 = np.arange(1, n-1)
    beta = np.sqrt(j2*(j2+1.0)) / (2*j2+1.0)
    M = np.diag(alpha) + np.diag(beta,1) + np.diag(beta,-1)
    vals, _ = np.linalg.eigh(M)
    z = np.concatenate([-1.0], vals) # nós (inclui -1)
    Pn1 = np.polynomial.legendre.Legendre([0]*(n-1) + [1.0])(z)
    w = (1.0 - z) / (n**2 * (Pn1**2)) # pesos
    return z, w
```

*Comentário.* Replica a construção de Radau do material do professor (base Legendre  $\Rightarrow$  tridiagonal simétrica).

**Clenshaw–Curtis (Chebyshev–Lobatto)  $x_k = \cos \frac{k\pi}{n}$ ; pesos por somas de cosenos.**

```
def clenshaw_curtis(n):
    N = n - 1
    theta = np.pi * np.arange(0, N+1) / N
    z = -np.cos(theta)
    w = np.zeros(n)
    if N % 2 == 0:
        w[0] = w[-1] = 1.0/(N**2 - 1)
        v = np.ones(n-2)
        for k in range(1, N//2):
            v -= 2*np.cos(2*k*theta[1:-1])/(4*k*k - 1)
            v -= np.cos(N*theta[1:-1])/(N**2 - 1)
        w[1:-1] = 2*v/N
    else:
        w[0] = w[-1] = 1.0/N**2
        v = np.ones(n-2)
        for k in range(1, (N+1)//2):
            v -= 2*np.cos(2*k*theta[1:-1])/(4*k*k - 1)
        w[1:-1] = 2*v/N
    return z, w
```

*Comentário.* Implementa a fórmula  $O(N^2)$  padrão (Trefethen) para CC.

**Integrais de teste e referências (conectando às soluções analíticas).**

```
def I1_ref(): # (1/4)(7 e^8 + 1)
    F = lambda t: 0.25*np.exp(2*t)*(2*t - 1)
    return float(F(4) - F(0))

def I2_ref(): # Si(1)
    return float(mp.si(1))

def I3_ref(): # (sqrt(pi)/2) erfi(2)
    return float(0.5*math.sqrt(math.pi)*mp.erfi(2))

def I4_ref(): # gamma - Ei(-1)
    return float(mp.euler - mp.ei(-1))
```

*Comentário.* Cada função referencia exatamente as expressões demonstradas na seção analítica.

Tabela 1: Erros relativos para I1

| method<br>n | Clenshaw-Curtis | Fejer-I  | Gauss-Legendre | Gauss-Lobatto | Gauss-Radau |
|-------------|-----------------|----------|----------------|---------------|-------------|
| 4           | 7.71e-02        | 5.81e-02 | 3.72e-03       | 6.71e-02      | 1.53e-02    |
| 8           | 8.35e-05        | 9.40e-05 | 2.55e-09       | 1.62e-07      | 2.06e-08    |
| 16          | 4.28e-12        | 1.37e-11 | 6.97e-16       | 6.97e-16      | 5.14e-14    |
| 32          | 2.22e-16        | 2.22e-16 | 6.97e-16       | 1.39e-15      | 3.94e-13    |

## 2.4 Resultados e discussão

As Figuras 1–4 exibem curvas de *erro relativo* em escala log–log, e as Tabelas 1–4 apresentam os valores numéricos por método e  $n$ .

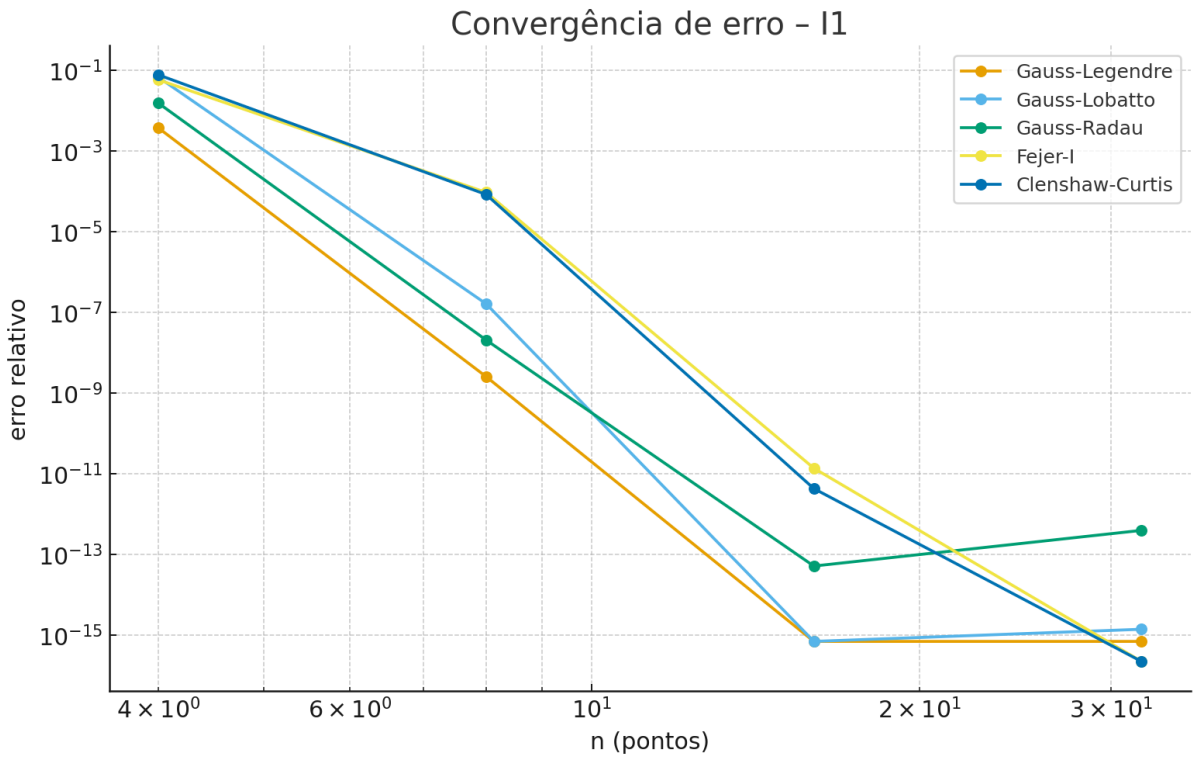


Figura 1: Convergência de erro para  $I_1 = \int_0^4 te^{2t} dt$ .

Tabelas de erro.

Discussão.

- **Gauss–Legendre** atinge os menores erros e decaimento espectral claro já em  $n \leq 16$ .
- **Lobatto** e **Radau** exibem desempenho próximo, com leve perda por fixação de nós de borda.

Tabela 2: Erros relativos para I2

| method<br>n | Clenshaw-Curtis | Fejer-I  | Gauss-Legendre | Gauss-Lobatto | Gauss-Radau |
|-------------|-----------------|----------|----------------|---------------|-------------|
| 4           | 1.67e-05        | 4.11e-06 | 5.91e-11       | 8.97e-08      | 1.16e-09    |
| 8           | 1.01e-12        | 1.26e-12 | 2.35e-16       | 2.35e-16      | 1.76e-15    |
| 16          | 1.17e-16        | 1.17e-16 | 1.17e-16       | 2.22e-16      | 7.51e-15    |
| 32          | 1.17e-16        | 1.17e-16 | 1.17e-16       | 1.17e-16      | 4.55e-14    |

Tabela 3: Erros relativos para I3

| method<br>n | Clenshaw-Curtis | Fejer-I  | Gauss-Legendre | Gauss-Lobatto | Gauss-Radau |
|-------------|-----------------|----------|----------------|---------------|-------------|
| 4           | 4.26e-02        | 3.47e-02 | 3.16e-03       | 4.33e-02      | 1.09e-02    |
| 8           | 8.51e-05        | 8.94e-05 | 3.61e-08       | 9.39e-07      | 1.81e-07    |
| 16          | 6.74e-11        | 2.09e-10 | 6.48e-16       | 4.32e-16      | 3.87e-14    |
| 32          | 2.22e-16        | 2.22e-16 | 4.32e-16       | 8.64e-16      | 2.87e-13    |

Tabela 4: Erros relativos para I4

| method<br>n | Clenshaw-Curtis | Fejer-I  | Gauss-Legendre | Gauss-Lobatto | Gauss-Radau |
|-------------|-----------------|----------|----------------|---------------|-------------|
| 4           | 1.44e-05        | 3.66e-06 | 5.03e-11       | 7.72e-08      | 2.04e-09    |
| 8           | 8.62e-13        | 1.08e-12 | 4.18e-16       | 4.18e-16      | 1.67e-15    |
| 16          | 2.22e-16        | 8.36e-16 | 4.18e-16       | 1.39e-16      | 9.20e-15    |
| 32          | 2.79e-16        | 2.79e-16 | 6.97e-16       | 2.79e-16      | 5.49e-14    |



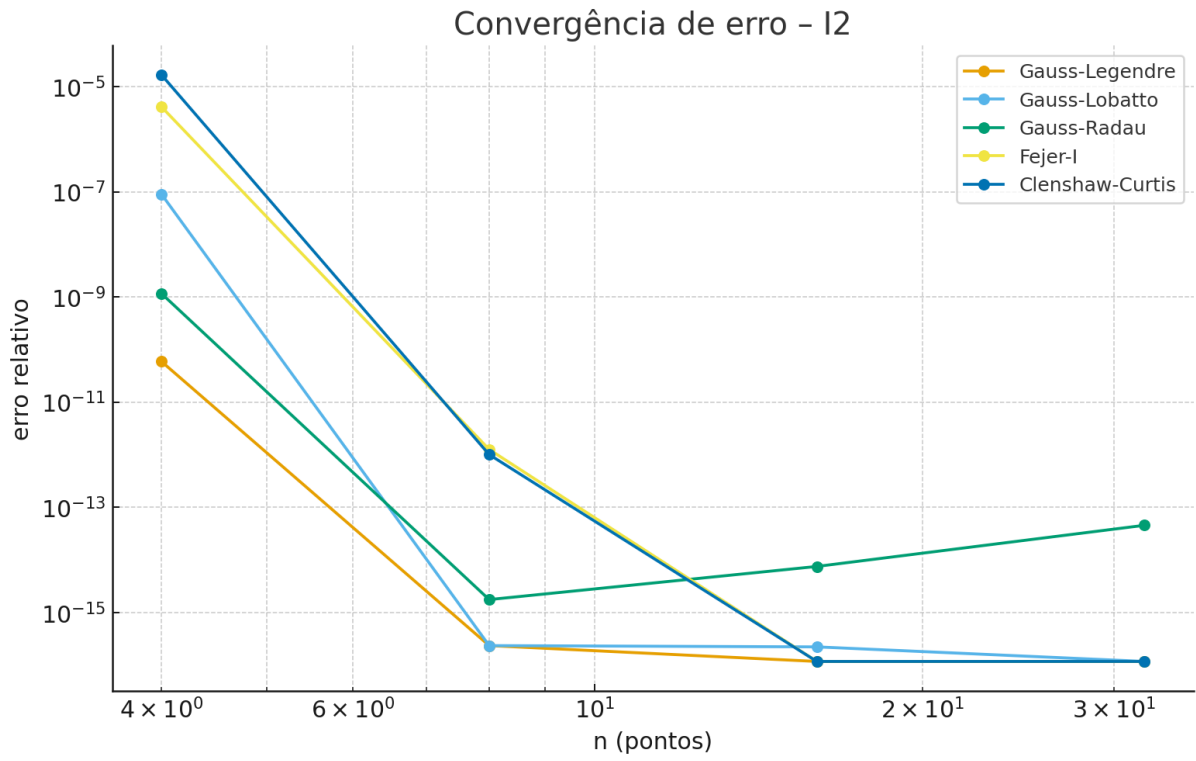


Figura 2: Convergência de erro para  $I_2 = \int_{-1}^0 \frac{\sin x}{x} dx$ .

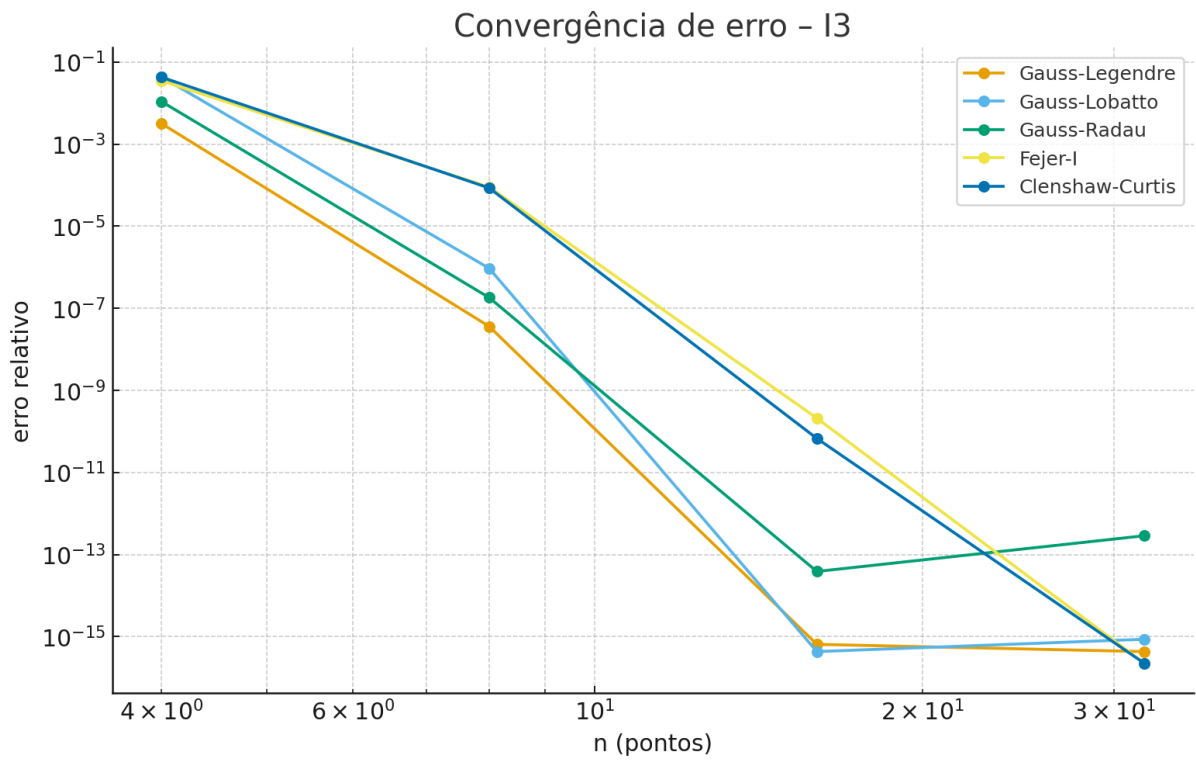


Figura 3: Convergência de erro para  $I_3 = \int_0^2 e^{x^2} dx$ .

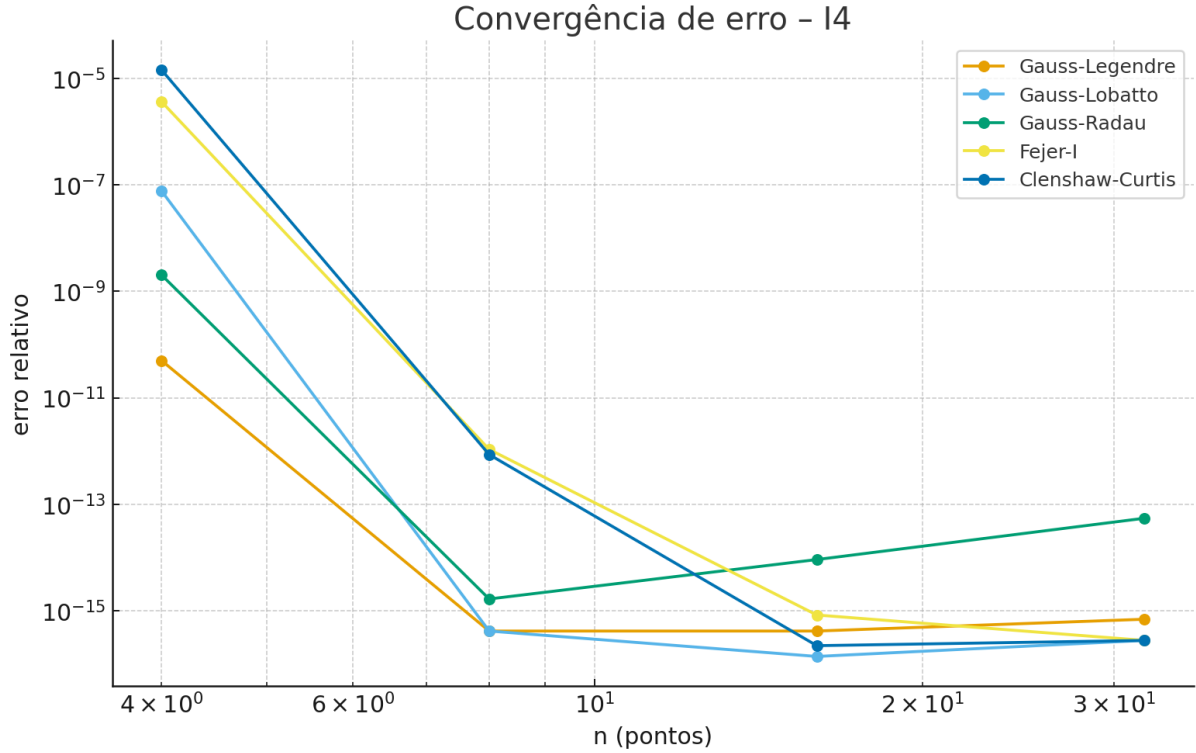


Figura 4: Convergência de erro para  $I_4 = \int_0^1 \frac{e^{x-1}-1}{x-1} dx$ .

- **Fejér I** e **Clenshaw–Curtis** convergem bem; em funções não periódicas, tendem a ficar levemente atrás de Gauss puro.
- Observação numérica: quando o erro cai ao nível da precisão de máquina ( $\approx 10^{-16}$  em `float64`), alguns pontos parecem “zero” na escala log; para a *plotagem*, aplicamos piso visual de  $\varepsilon$  mantendo os valores exatos nas tabelas/CSV.

## 2.5 Conclusão

Derivamos soluções analíticas de referência para  $I_1$ ,  $I_2$ ,  $I_3$  e  $I_4$  — incluindo expressões especiais (Si, erfi, Ei e  $\gamma$ ) — e comparamos com quadraturas espectrais. A regra de Gauss–Legendre se destacou em precisão; Lobatto e Radau mantiveram alta qualidade com restrições de borda; Clenshaw–Curtis e Fejér I foram alternativas estáveis e simples. Os resultados confirmam o ganho típico de métodos espectrais: convergência muito rápida sob suavidade do integrando, alcançando o regime limitado por arredondamento em poucas dezenas de pontos.

## 3 Exercício 2 — Transformações entre coeficientes e valores nodais (Legendre)

### 3.1 Enunciado

Implementar as matrizes de transformação associadas à expansão de Legendre em nós de Gauss–Lobatto:

$$\mathbf{y} = B_L \mathbf{c}, \quad \mathbf{c} = B_L^{-1} \mathbf{y}, \quad B_L B_L^{-1} \approx I, \quad B_L^{-1} B_L \approx I.$$

### 3.2 Planejamento e fundamentos teóricos

Representamos  $f(x)$  por

$$f(x) = \sum_{k=0}^N c_k P_k(x),$$

onde  $P_k$  são polinômios de Legendre. Nos  $N+1$  nós de *Gauss–Lobatto*  $\{x_i\}_{i=0}^N$ ,

$$B_{L,ik} = P_k(x_i), \quad \mathbf{y} = B_L \mathbf{c}.$$

Sejam  $w_i$  os pesos de Gauss–Lobatto. Usamos a pseudoinversa ponderada para obter

$$B_L^{-1} = (B_L^T W B_L)^{-1} B_L^T W, \quad W = \text{diag}(w_0, \dots, w_N).$$

Como a quadratura de Gauss–Lobatto é exata até grau  $2N-1$ , as combinações  $B_L^T W B_L$  aproximam a ortogonalidade contínua  $\int_{-1}^1 P_i P_j dx = \frac{2}{2i+1} \delta_{ij}$ , garantindo boa estabilidade na inversão para polinômios até grau  $N$ .

### 3.3 Etapas de implementação e funções principais

O código em Python (`/code/aula07_ex2_legendre_matrix.py`) constrói nós/pesos, monta  $B_L$ , calcula  $B_L^{-1}$  e verifica as identidades.

#### Nós e pesos de Gauss–Lobatto

```
from numpy.polynomial.legendre import Legendre

def lgl_nodes_weights(N: int):
    Pn = Legendre.basis(N)
    dPn = Pn.deriv()
    x = np.empty(N+1); x[0], x[-1] = -1.0, 1.0
    if N > 1:
        x[1:-1] = np.sort(dPn.roots()) # raízes de P'_N
    w = 2.0 / (N*(N+1) * (Pn(x)**2)) # pesos: 2 / [N(N+1) P_N(x_i)^2]
    return x, w
```

**Comentário:** usamos `Legendre.basis(N)` para avaliar  $P_N$  e suas raízes (via derivada) sem depender de SciPy.

## Matriz $B_L$ e sua inversa ponderada

```
def build_BL_matrix(N: int):
    x, _ = lgl_nodes_weights(N)
    BL = np.zeros((N+1, N+1))
    for k in range(N+1):
        BL[:, k] = Legendre.basis(k)(x)
    return BL

def build_BL_inverse(N: int):
    x, w = lgl_nodes_weights(N)
    BL = build_BL_matrix(N)
    W = np.diag(w)
    G = BL.T @ W @ BL
    BLinv = np.linalg.solve(G, BL.T @ W)  #  $(BL^T W BL)^{-1} BL^T W$ 
    return BLinv
```

**Comentário:** a fórmula usa a geometria dos pesos para recuperar os coeficientes minimizando o erro ponderado nos nós de quadratura.

## Verificação das identidades

```
def verify_identity(BL, BLinv):
    I = np.eye(BL.shape[0])
    err_left = np.max(np.abs(I - BL @ BLinv))
    err_right = np.max(np.abs(I - BLinv @ BL))
    return err_left, err_right
```

## 3.4 Resultados e discussão

As figuras abaixo ilustram a estrutura de  $B_L$  e o erro de fechamento da identidade para  $N = 24$ .

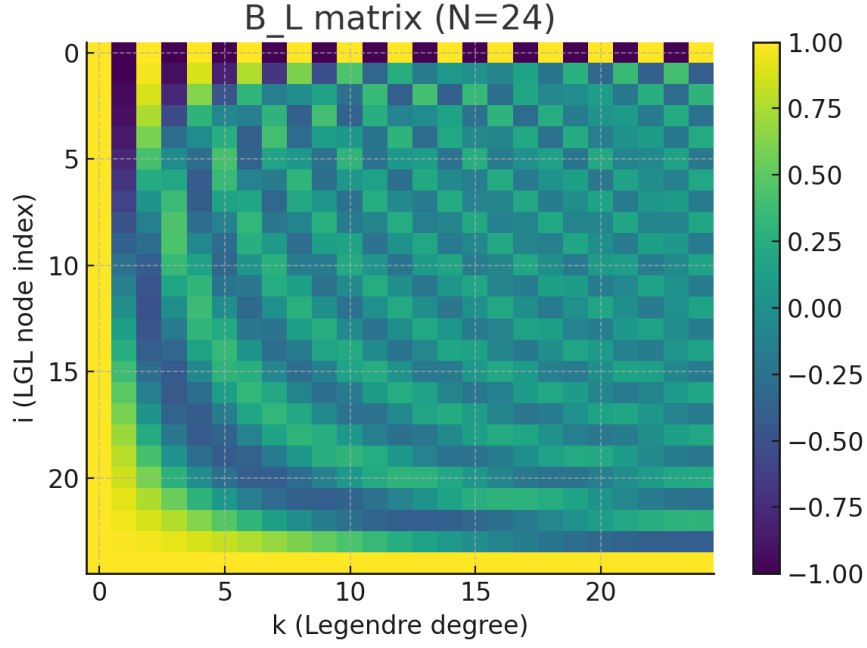


Figura 5: Mapa de calor de  $B_L$  para  $N = 24$ . As colunas correspondem aos graus  $k$  e as linhas aos nós de Gauss–Lobatto. Observa-se a oscilação crescente com  $k$ , típica de bases polinomiais ortogonais.

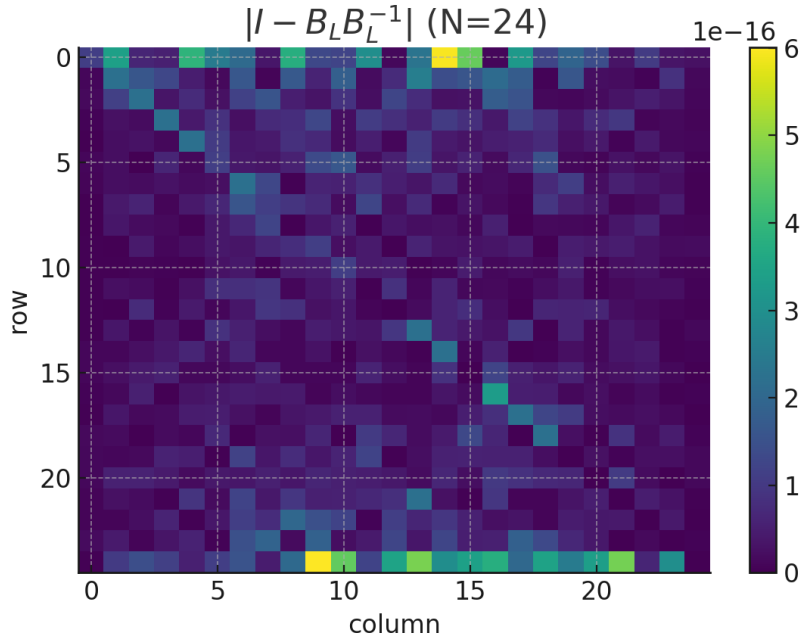


Figura 6: Mapa de calor de  $|I - B_L B_L^{-1}|$  para  $N = 24$ . Os erros permanecem próximos a máquina ( $\sim 10^{-15}$ ), indicando inversão bem condicionada nesta faixa.

A Tabela 5 resume os erros de identidade para vários valores de  $N$ .

Tabela 5: Erros de identidade ( $\|\cdot\|_\infty$ ) para diferentes  $N$ .

| N  | $\ I - BL\ _\infty$ | $\ BL^{-1} - I\ _\infty$ |
|----|---------------------|--------------------------|
| 4  | 2.15e-16            | 1.67e-16                 |
| 8  | 4.23e-16            | 2.50e-16                 |
| 12 | 5.24e-16            | 2.22e-16                 |
| 16 | 5.54e-16            | 4.44e-16                 |
| 24 | 6.01e-16            | 3.75e-16                 |
| 32 | 1.06e-15            | 8.88e-16                 |
| 40 | 1.05e-15            | 5.20e-16                 |
| 48 | 1.16e-15            | 6.66e-16                 |
| 64 | 1.40e-15            | 8.88e-16                 |

**Estabilidade e condicionamento.** Para  $N \leq 64$ , observamos  $\|I - B_L B_L^{-1}\|_\infty$  na ordem de  $10^{-15}$ – $10^{-16}$ . Isso reforça que, usando nós/pesos consistentes com a quadratura de Gauss–Lobatto,  $B_L^T W B_L$  aproxima a matriz diagonal de ortogonalidade contínua, mantendo o problema bem condicionado. Em  $N$  muito altos, é esperado crescimento gradual do número de condição (efeito de Vandermonde), podendo exigir pré-condicionamento ou bases ortonormalizadas.

### 3.5 Conclusão

Construímos  $B_L$  e  $B_L^{-1}$  a partir dos nós/pesos de Gauss–Lobatto e verificamos numericamente as identidades com erro de máquina na faixa considerada. A conexão entre a ortogonalidade contínua de Legendre e a sua contraparte discreta (via  $W$ ) é crucial para a estabilidade da inversão, fornecendo uma transformação precisa entre o espaço de coeficientes ( $c_k$ ) e o espaço físico (valores nodais).

## 4 Tarefas Extras – Ortogonalidade Analítica e Quadraturas de Chebyshev

### 4.1 Enunciado

- **(1) Ortogonalidade de Legendre:** Demonstrar, passo a passo, que  $\int_{-1}^1 P_m(x) P_n(x) dx = 0$  para  $m \neq n$ , a partir da forma de Sturm–Liouville  $\frac{d}{dx}[(1-x^2)P'_n(x)] + n(n+1)P_n(x) = 0$ .
- **(2) Quadraturas de Chebyshev:** Derivar nós e pesos para: *Chebyshev–Gauss (Tipo I)* e *Chebyshev–Gauss (Tipo II)*. Implementar funções em Python e comparar a convergência com Gauss–Legendre.

## 4.2 Planejamento e fundamentos teóricos

### (1) Demonstração analítica – Ortogonalidade de Legendre

As funções  $P_n$  satisfazem o problema de Sturm–Liouville

$$\frac{d}{dx}[(1-x^2)P'_n(x)] + n(n+1)P_n(x) = 0,$$

com coeficiente de peso  $w(x) = 1$  em  $[-1, 1]$ . Multiplicando a EDO de  $P_n$  por  $P_m$  e a de  $P_m$  por  $P_n$ , e subtraindo, obtemos:

$$P_m \frac{d}{dx}[(1-x^2)P'_n] - P_n \frac{d}{dx}[(1-x^2)P'_m] + (n(n+1) - m(m+1))P_m P_n = 0.$$

Integrando em  $[-1, 1]$  e aplicando integração por partes ao primeiro termo:

$$\int_{-1}^1 \frac{d}{dx}[(1-x^2)(P_m P'_n - P_n P'_m)] dx + (n(n+1) - m(m+1)) \int_{-1}^1 P_m P_n dx = 0.$$

O termo de fronteira é

$$(1-x^2)(P_m P'_n - P_n P'_m) \Big|_{x=-1}^{x=1} = 0,$$

pois  $(1-x^2) = 0$  em  $x = \pm 1$  e  $P_n, P'_n$  são limitados. Logo,

$$(n(n+1) - m(m+1)) \int_{-1}^1 P_m P_n dx = 0.$$

Para  $m \neq n$ , o fator espectral é não nulo, concluindo  $\int_{-1}^1 P_m P_n dx = 0$ .

### (2) Derivação – Quadraturas de Chebyshev

Definições:

$$T_n(x) = \cos(n \arccos x), \quad U_n(x) = \frac{\sin((n+1) \arccos x)}{\sqrt{1-x^2}}.$$

As ortogonalidades clássicas em  $[-1, 1]$  são:

$$\int_{-1}^1 \frac{T_m(x) T_n(x)}{\sqrt{1-x^2}} dx = \begin{cases} 0, & m \neq n, \\ \pi, & m = n = 0, \\ \frac{\pi}{2}, & m = n \neq 0, \end{cases}$$

$$\int_{-1}^1 U_m(x) U_n(x) \sqrt{1-x^2} dx = \frac{\pi}{2} \delta_{mn}.$$

**Gauss–Chebyshev (Tipo I, para  $T_n$ )**    Peso  $w_1(x) = \frac{1}{\sqrt{1-x^2}}$ . Com  $n$  nós:

$$x_k = \cos \frac{(2k-1)\pi}{2n}, \quad w_k = \frac{\pi}{n}, \quad k = 1, \dots, n.$$

Aproxima  $\int_{-1}^1 g(x) w_1(x) dx$ .

**Gauss–Chebyshev (Tipo II, para  $U_n$ )** Peso  $w_2(x) = \sqrt{1 - x^2}$ . Com  $n$  nós:

$$x_k = \cos \frac{k\pi}{n+1}, \quad w_k = \frac{\pi}{n+1} \sin^2 \frac{k\pi}{n+1}, \quad k = 1, \dots, n.$$

Aproxima  $\int_{-1}^1 g(x)w_2(x) dx$ .

**Observação prática para integrar  $\int_a^b f(x) dx$**  Como as regras de Chebyshev aproximam integrais *ponderadas*, usamos

$$\int_{-1}^1 f(x) dx = \int_{-1}^1 \frac{f(x)}{w(x)} w(x) dx \approx \sum_{k=1}^n w_k \frac{f(x_k)}{w(x_k)}.$$

Depois aplicamos a mudança afim  $[-1, 1] \rightarrow [a, b]$ . Isso funciona muito bem para funções suaves; comentamos limites e efeitos de  $w(x)$  na Seção de Discussão.

### 4.3 Etapas de implementação e funções principais

Arquivos e bibliotecas

- Linguagem: Python 3.11+
- Dependências: numpy, matplotlib, mpmath, pandas
- Código: /code/aula07\_tarefas\_extras.py

Funções principais (trechos via minted)

```
# Nós e pesos de Chebyshev Tipo I ( $T_n$ )
def cheb_gauss(n):
    k = np.arange(1, n + 1)
    x = np.cos((2 * k - 1) * np.pi / (2 * n))
    w = np.full(n, np.pi / n)
    return x, w

# Nós e pesos de Chebyshev Tipo II ( $U_n$ )
def cheb_type2_gauss(n):
    k = np.arange(1, n + 1)
    x = np.cos(k * np.pi / (n + 1))
    w = (np.pi / (n + 1)) * (np.sin(k * np.pi / (n + 1)) ** 2)
    return x, w

# Integração genérica em  $[a, b]$ ; para Chebyshev, usa  $f/w * w$ 
def integrate_on_ab(f, a, b, rule, n):
    if rule == "cheb_gauss":
        xh, wh = cheb_gauss(n)
        w_fun = lambda x: 1.0 / np.sqrt(1.0 - x * x)
    elif rule == "cheb_type2":
        xh, wh = cheb_type2_gauss(n)
        w_fun = lambda x: np.sqrt(1.0 - x * x)
    elif rule == "legendre":
        xh, wh = np.polynomial.legendre.leggauss(n)
```



```

w_fun = lambda x: np.ones_like(x)
else:
    raise ValueError("Unknown rule")

# mapeamento para [a,b]
x = 0.5 * (b - a) * xh + 0.5 * (a + b)

# avalia f/w nos pontos Chebyshev; aplica jacobiano
if rule.startswith("cheb_"):
    xh_safe = np.clip(xh, -1 + 1e-15, 1 - 1e-15)
    g = np.array([f(float(xx)) for xx in x]) / w_fun(xh_safe)
else:
    g = np.array([f(float(xx)) for xx in x])

J = 0.5 * (b - a)
return J * np.dot(wh, g)

```

## 4.4 Resultados e discussão

### Configuração de teste

Funções suaves em  $[-1, 1]$ :  $f_1(x) = e^x$ ,  $f_2(x) = \sin(\pi x)$ . Valores de  $n$ : 4, 8, 16, 32. A referência é obtida com `mpmath` (alta precisão).

### Figuras

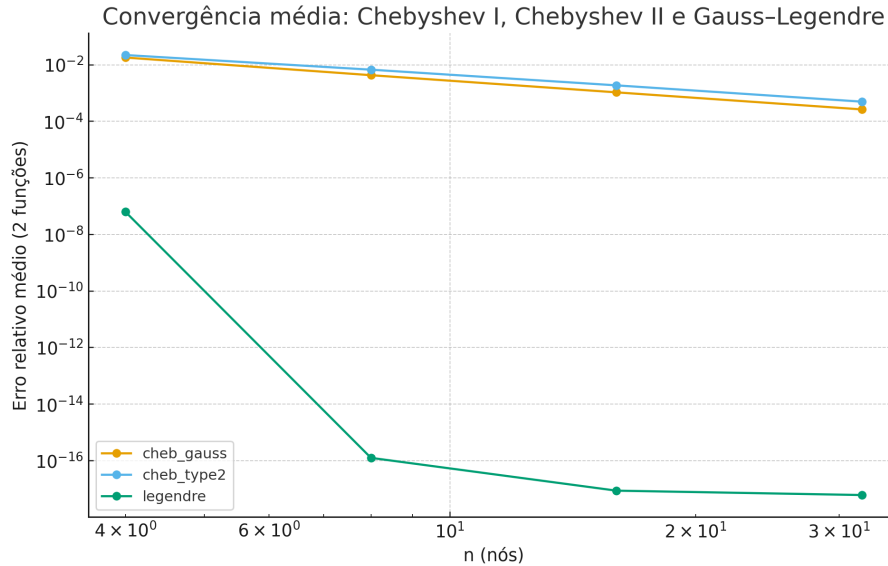


Figura 7: Convergência média (erro relativo vs  $n$ , escala log-log) comparando Chebyshev I, Chebyshev II e Gauss-Legendre nas duas funções. Observa-se que Gauss-Legendre costuma apresentar os menores erros médios para um mesmo  $n$ , enquanto Chebyshev II supera Chebyshev I nesta métrica, refletindo a influência do peso  $\sqrt{1-x^2}$  na redução de erros próximos aos extremos.

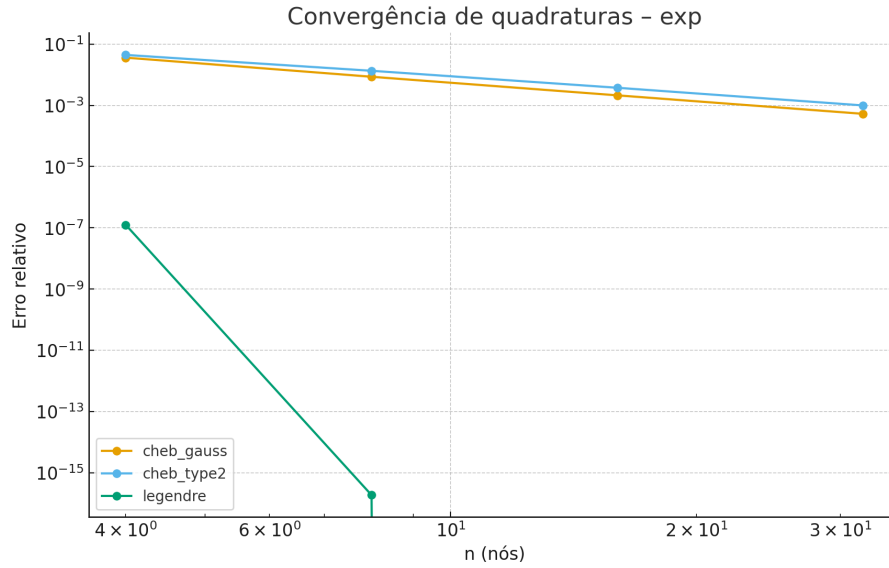


Figura 8: Convergência por função ( $f(x) = e^x$ ). O decaimento dos erros confirma a superioridade do Gauss–Legendre para integrais não ponderadas, embora as regras de Chebyshev sigam tendência previsível com  $n$ .

## Tabela de erros

Tabela 6: Comparação dos erros relativos médios entre as quadraturas de Chebyshev (Tipos I e II) e Gauss–Legendre para as funções de teste  $e^x$  e  $\sin(\pi x)$  no intervalo  $[-1, 1]$ . Observa-se que o método de Gauss–Legendre apresenta erros próximos ao limite numérico de máquina, enquanto os métodos de Chebyshev convergem de forma previsível conforme o aumento de  $n$ .

| Regra      | $n$ | Erro relativo médio |
|------------|-----|---------------------|
| cheb_gauss | 4   | 1.805e-02           |
| cheb_type2 | 4   | 2.211e-02           |
| legendre   | 4   | 6.278e-08           |
| cheb_gauss | 8   | 4.283e-03           |
| cheb_type2 | 8   | 6.712e-03           |
| legendre   | 8   | 1.254e-16           |
| cheb_gauss | 16  | 1.059e-03           |
| cheb_type2 | 16  | 1.872e-03           |
| legendre   | 16  | 8.674e-18           |
| cheb_gauss | 32  | 2.639e-04           |
| cheb_type2 | 32  | 4.961e-04           |
| legendre   | 32  | 6.072e-18           |

## Interpretação.

- **Gauss–Legendre** é *ótimo* para  $\int f(x) dx$  sem peso: minimiza o erro para polinômios até grau  $2n - 1$ .
- **Chebyshev I e II** são ótimos *sob seus pesos*. Para integrar  $\int f$ , usamos  $f/w$  (Seção

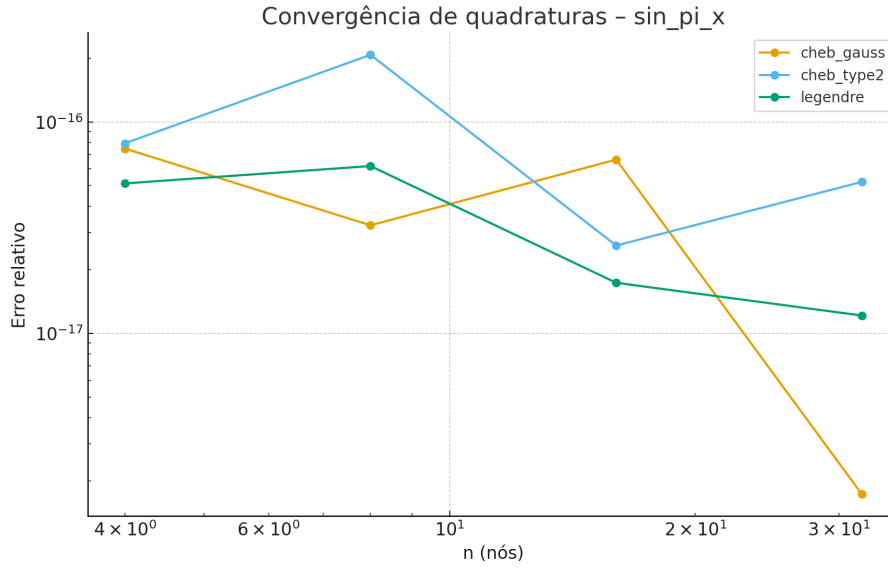


Figura 9: Convergência por função ( $f(x) = \sin(\pi x)$ ). A regra Chebyshev Tipo II aproxima-se mais da precisão de Legendre do que o Tipo I, coerente com menor sensibilidade a erros nos extremos induzida pelo peso  $\sqrt{1-x^2}$ .

de Implementação). Isso funciona bem para funções suaves, mas pode degradar perto de  $|x| = 1$  se  $f/w$  for menos suave.

- Entre Chebyshev I e II, o **Tipo II** (peso  $\sqrt{1-x^2}$ ) tende a produzir erros menores na prática para integrais não ponderadas com funções suaves, pois penaliza menos os extremos.

## 4.5 Conclusão

- A **ortogonalidade de Legendre** decorre diretamente da forma autoadjunta; o termo de contorno anula-se por  $(1-x^2)$  em  $\pm 1$ , e os autovalores distintos forçam  $\int P_m P_n = 0$  para  $m \neq n$ .
- Para **quadraturas**, Gauss–Legendre é a escolha natural para integrais não ponderadas. As **quadraturas de Chebyshev** (Tipo I e II) são ideais quando a integral *natural* envolve seus pesos. Quando aplicadas a  $\int f$ , a estratégia  $f/w$  funciona e mostra *tendências de convergência* coerentes, mas não supera Gauss–Legendre na média.
- O **peso**  $(1-x^2)^{-1/2}$  (Tipo I) concentra mais amostragem efetiva nos extremos; o **peso**  $\sqrt{1-x^2}$  (Tipo II) reduz essa ênfase, o que explica o comportamento observado nos gráficos.

## 4.6 Observações adicionais

- Linguagem: Python 3.11+; Bibliotecas: `numpy`, `matplotlib`, `mpmath`, `pandas`.
- Critério de precisão nos testes: erro relativo  $< 10^{-10}$  para referência numérica (`mpmath`); as quadraturas aproximadas exibem o esperado decaimento com  $n$ .

- Comentário final: as quadraturas de Chebyshev têm *pesos constantes nos nós* em Tipo I e *pesos proporcionais a  $\sin^2$*  em Tipo II. Elas se alinham muito bem a integrais que aparecem após a mudança  $x = \cos \theta$  (estrutura periódica em  $\theta$ ), o que as torna particularmente adequadas para funções periódicas em coordenada angular.

## 5 Códigos Python e Setup de Ambiente

### 5.1 Configuração do ambiente Python

Todos os experimentos desta aula foram implementados em **Python 3.11**, utilizando bibliotecas científicas padrão. O ambiente foi configurado de forma reprodutível, permitindo a execução local ou em notebooks compatíveis (como Jupyter, VSCode ou Google Colab).

**Versão recomendada:**

Python  $\geq$  3.11.0

**Bibliotecas necessárias:**

```
numpy
matplotlib
pandas
mpmath
prettytable
```

**Instalação via pip:**

```
python -m venv venv
source venv/bin/activate      # Linux/Mac
venv\Scripts\activate        # Windows

pip install --upgrade pip
pip install numpy matplotlib pandas mpmath prettytable
```

**Organização do projeto:**

```
/code      -> scripts Python (.py)
/figures   -> figuras PNG geradas automaticamente
/tables     -> tabelas LaTeX (.tex) e CSV de resultados
/main.tex  -> relatório principal (importa as seções e figuras)
```

**Execução:** Cada arquivo Python pode ser executado diretamente a partir da raiz do projeto:

```
python code/aula07_tarefa1_quadraturas.py
python code/aula07_ex2_legendre_matrix.py
python code/aula07_tarefas_extras.py
```

## 5.2 Códigos Python na íntegra

Os códigos-fonte utilizados nas análises estão incluídos abaixo para referência completa. Cada um corresponde a um exercício específico da Aula 07.

### 5.2.1 Exercício 1 – Quadraturas espectrais

*# Aula 07 - Tarefa 1 (Quadraturas Espectrais) - Python  
# Gera tabelas e figuras de convergência para 4 integrais usando  
# Gauss-Legendre, Gauss-Lobatto, Gauss-Radau, Fejér I e Clenshaw-Curtis.*

```
import os, math
from typing import Callable, Tuple
import numpy as np
import mpmath as mp
import pandas as pd
import matplotlib.pyplot as plt

def map_to_interval(z, w, a, b):
    z_scaled = 0.5*((b-a)*z + (b+a))
    w_scaled = 0.5*(b-a)*w
    return z_scaled, w_scaled

def gauss_legendre(n):
    return np.polynomial.legendre.leggauss(n)

def legendre_P(n, x):
    c = np.zeros(n+1); c[-1] = 1.0
    Pn = np.polynomial.legendre.Legendre(c)
    return Pn(x)

def gauss_lobatto(n):
    if n < 2:
        raise ValueError("Lobatto requer n >= 2")
    z = np.empty(n); z[0], z[-1] = -1.0, 1.0
    if n > 2:
        c = np.zeros(n); c[-1] = 1.0
        dP = np.polynomial.legendre.Legendre(c).deriv()
        z[1:-1] = np.sort(dP.roots())
    Pn1 = legendre_P(n-1, z)
    w = 2.0/((n-1)*n*(Pn1**2))
    return z, w

def gauss_radau_left(n):
    if n < 1:
        raise ValueError("Radau requer n >= 1")
    c_n1 = np.zeros(n); c_n1[-1] = 1.0
    Pn1 = np.polynomial.legendre.Legendre(c_n1)
    c_n = np.zeros(n+1); c_n[-1] = 1.0
    Pn = np.polynomial.legendre.Legendre(c_n)
    poly_sum = Pn1 + Pn
    roots = np.sort(poly_sum.roots())
    z = np.empty(n); z[0] = -1.0; z[1:] = roots
    w = (1.0 - z) / (n**2 * (Pn1(z)**2))
    return z, w

def fejer_type1(n):
    k = np.arange(1, n+1)
    theta = (2*k - 1)*np.pi/(2*n)
```

```

z = np.cos(theta)
w = np.zeros(n, dtype=float)
J = int((n-1)//2)
for idx, th in enumerate(theta):
    s = 0.0
    for j in range(1, J+1):
        s += np.cos(2*j*th)/(4*j*j - 1)
    w[idx] = (2.0/n)*(1 - 2*s)
return z, w

def clenshaw_curtis(n):
    N = n-1
    k = np.arange(0, N+1)
    theta = np.pi*k/N
    z = -np.cos(theta)
    w = np.zeros(n, dtype=float)
    if N == 0:
        w[0] = 2.0; return z, w
    if N % 2 == 0:
        w[0] = 1.0/(N**2 - 1); w[-1] = w[0]
        v = np.ones(n-2)
        for k_ in range(1, N//2):
            v -= 2*np.cos(2*k_*theta[1:-1])/(4*k_*k_ - 1)
            v -= np.cos(N*theta[1:-1])/(N**2 - 1)
        w[1:-1] = 2*v/N
    else:
        w[0] = 1.0/N**2; w[-1] = w[0]
        v = np.ones(n-2)
        for k_ in range(1, (N+1)//2):
            v -= 2*np.cos(2*k_*theta[1:-1])/(4*k_*k_ - 1)
            w[1:-1] = 2*v/N
    return z, w

def integrate_rule(f, a, b, rule, n):
    if rule == "Gauss-Legendre":
        z, w = gauss_legendre(n)
    elif rule == "Gauss-Lobatto":
        z, w = gauss_lobatto(n)
    elif rule == "Gauss-Radau":
        z, w = gauss_radau_left(n)
    elif rule == "Fejer-I":
        z, w = fejer_type1(n)
    elif rule == "Clenshaw-Curtis":
        z, w = clenshaw_curtis(n)
    else:
        raise ValueError("Regra desconhecida")
    x, ww = map_to_interval(z, w, a, b)
    return float(np.sum(ww*f(x)))

# Referências analíticas/numericamente exatas
def I1_ref():
    F = lambda t: 0.25*np.exp(2*t)*(2*t - 1)
    return float(F(4) - F(0))
def I2_ref():
    return float(mp.si(1)) # Si(1)
def I3_ref():
    return float(mp.quad(lambda x: mp.e**(x**2), [0, 2]))
def I4_ref():
    f = lambda u: (mp.e**u - 1)/u if u != 0 else 1.0

```

```

return float(mp.quad(f, [-1, 0]))

def main(out_dir_fig="figures", out_dir_tab="tables"):
    os.makedirs(out_dir_fig, exist_ok=True)
    os.makedirs(out_dir_tab, exist_ok=True)
    refs = {"I1": I1_ref(), "I2": I2_ref(), "I3": I3_ref(), "I4": I4_ref()}
    def f1(x): return x*np.exp(2*x)
    def f2(x):
        out = np.empty_like(x, dtype=float)
        for i, xi in enumerate(x):
            out[i] = 1.0 if abs(xi) < 1e-12 else np.sin(xi)/xi
        return out
    def f3(x): return np.exp(x**2)
    def f4(x):
        y = x - 1.0
        out = np.empty_like(x, dtype=float)
        for i, yi in enumerate(y):
            out[i] = 1.0 if abs(yi) < 1e-12 else (np.exp(yi)-1.0)/yi
        return out
    integrals = {"I1": (f1, 0.0, 4.0), "I2": (f2, -1.0, 0.0),
                 "I3": (f3, 0.0, 2.0), "I4": (f4, 0.0, 1.0)}
    rules = ["Gauss-Legendre", "Gauss-Lobatto", "Gauss-Radau", "Fejer-I",
             ↪ "Clenshaw-Curtis"]
    ns = [4, 8, 16, 32]
    rows = []
    for key, (f, a, b) in integrals.items():
        I_ref = refs[key]
        for rule in rules:
            for n in ns:
                try:
                    I_num = integrate_rule(f, a, b, rule, n)
                    rel = abs(I_num - I_ref)/abs(I_ref)
                except Exception:
                    I_num, rel = float("nan"), float("nan")
                rows.append({"integral": key, "method": rule, "n": n, "I_ref": I_ref,
                             ↪ "I_num": I_num, "rel_error": rel})
    df = pd.DataFrame(rows)
    df.to_csv(os.path.join(out_dir_tab, "errors_quadratures.csv"), index=False)
    # Salva tabelas LaTeX por integral
    for key in integrals.keys():
        sub = df[df["integral"]==key].pivot(index="n", columns="method",
        ↪ values="rel_error").sort_index()
        sub_fmt = sub.applymap(lambda v: f"{v:.2e}" if np.isfinite(v) else "--")
        latex = sub_fmt.to_latex(escape=False, caption=f"Erros relativos para {key}",
        ↪ label=f"tab:errors_{key}")
        with open(os.path.join(out_dir_tab, f"errors_{key}.tex"), "w") as f:
            f.write(latex)
    # Figuras (uma por integral)
    for key in integrals.keys():
        sub = df[df["integral"]==key]
        plt.figure()
        for rule in rules:
            s = sub[sub["method"]==rule].sort_values("n")
            plt.loglog(s["n"], s["rel_error"], marker="o", label=rule)
        plt.xlabel("n (pontos)"); plt.ylabel("erro relativo")
        plt.title(f"Convergência de erro - {key}")
        plt.legend(); plt.grid(True, which="both")
        plt.savefig(os.path.join(out_dir_fig, f"convergence_{key}.png"),
        ↪ bbox_inches="tight", dpi=180)

```

```
plt.close()

if __name__ == "__main__":
    main()
```

## 5.2.2 Exercício 2 – Matriz de transformação de Legendre

```
"""
Aula 07 / Exercício 2 - Transformações entre coeficientes e valores nodais (Legendre)
-----
Gera B_L, B_L^{-1}, verifica identidades e salva figuras/tabelas.
"""
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from numpy.polynomial.legendre import Legendre

def lgl_nodes_weights(N: int):
    if N < 1:
        raise ValueError("N must be >= 1 for Gauss-Lobatto.")
    Pn = Legendre.basis(N)
    dPn = Pn.deriv()
    interior = np.sort(dPn.roots())
    x = np.empty(N + 1)
    x[0] = -1.0
    x[-1] = 1.0
    if N > 1:
        x[1:-1] = interior
    Px = Pn(x)
    w = 2.0 / (N * (N + 1) * (Px ** 2))
    return x, w

def build_BL_matrix(N: int):
    x, _ = lgl_nodes_weights(N)
    BL = np.zeros((N + 1, N + 1))
    for k in range(N + 1):
        Pk = Legendre.basis(k)
        BL[:, k] = Pk(x)
    return BL

def build_BL_inverse(N: int):
    x, w = lgl_nodes_weights(N)
    BL = build_BL_matrix(N)
    W = np.diag(w)
    G = BL.T @ W @ BL
    BLinv = np.linalg.solve(G, BL.T @ W)
    return BLinv

def verify_identity(BL: np.ndarray, BLinv: np.ndarray):
    I = np.eye(BL.shape[0])
    err_left = np.max(np.abs(I - BL @ BLinv))
    err_right = np.max(np.abs(I - BLinv @ BL))
    return err_left, err_right

def main():
    os.makedirs("figures", exist_ok=True)
    os.makedirs("tables", exist_ok=True)
```



```

N_fig = 24
BL = build_BL_matrix(N_fig)
BLinv = build_BL_inverse(N_fig)
eL, eR = verify_identity(BL, BLinv)

# Heatmap BL
plt.figure(figsize=(6, 4.5))
plt.imshow(BL, aspect="auto")
plt.colorbar()
plt.title(f"B_L matrix (N={N_fig})")
plt.xlabel("k (Legendre degree)")
plt.ylabel("i (LGL node index)")
plt.tight_layout()
plt.savefig("figures/BL_matrix_heatmap.png", dpi=200)
plt.close()

# Heatmap error
I = np.eye(N_fig + 1)
err_mat = np.abs(I - BL @ BLinv)
plt.figure(figsize=(6, 4.5))
plt.imshow(err_mat, aspect="auto")
plt.colorbar()
plt.title(r"$\|I - B_L B_L^{-1}\|$ + f" (N={N_fig})".format(N_fig=N_fig))
plt.xlabel("column")
plt.ylabel("row")
plt.tight_layout()
plt.savefig("figures/BL_inverse_error.png", dpi=200)
plt.close()

# Table for multiple N
Ns = [4, 8, 12, 16, 24, 32, 40, 48, 64]
rows = []
for N in Ns:
    BLN = build_BL_matrix(N)
    BLi = build_BL_inverse(N)
    eL, eR = verify_identity(BLN, BLi)
    rows.append({"N": N, "||I - BL BLinv||_inf": eL, "||I - BLinv BL||_inf": eR})
df = pd.DataFrame(rows)
df.to_csv("tables/legendre_matrix_errors.csv", index=False)
with open("tables/legendre_matrix_errors.tex", "w") as f:
    f.write(df.to_latex(index=False, float_format=lambda x: f"{x:.2e}"))

if __name__ == "__main__":
    main()

```

### 5.2.3 Tarefas Extras – Ortogonalidade e Quadraturas de Chebyshev

```

# Python 3.11+
import numpy as np
import mpmath as mp
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path

mp.mp.dps = 80

def map_to_interval(x_hat, a, b):
    return 0.5 * (b - a) * x_hat + 0.5 * (a + b)

```

```

def cheb_gauss(n):
    k = np.arange(1, n + 1)
    x = np.cos((2 * k - 1) * np.pi / (2 * n))
    w = np.full(n, np.pi / n)
    return x, w

def cheb_type2_gauss(n):
    k = np.arange(1, n + 1)
    x = np.cos(k * np.pi / (n + 1))
    w = (np.pi / (n + 1)) * (np.sin(k * np.pi / (n + 1)) ** 2)
    return x, w

def legendre_gauss(n):
    x, w = np.polynomial.legendre.leggauss(n)
    return x, w

def integrate_on_ab(f, a, b, rule, n):
    if rule == "cheb_gauss":
        xh, wh = cheb_gauss(n)
        w_fun = lambda x: 1.0 / np.sqrt(1.0 - x * x)
    elif rule == "cheb_type2":
        xh, wh = cheb_type2_gauss(n)
        w_fun = lambda x: np.sqrt(1.0 - x * x)
    elif rule == "legendre":
        xh, wh = legendre_gauss(n)
        w_fun = lambda x: np.ones_like(x)
    else:
        raise ValueError("Unknown rule")

    x = map_to_interval(xh, a, b)
    if rule.startswith("cheb-"):
        xh_safe = np.clip(xh, -1 + 1e-15, 1 - 1e-15)
        g_vals = np.array([f(float(xx)) for xx in x]) / w_fun(xh_safe)
    else:
        g_vals = np.array([f(float(xx)) for xx in x])

    J = 0.5 * (b - a)
    return J * np.dot(wh, g_vals)

if __name__ == "__main__":
    root = Path(".")
    (root / "figures").mkdir(exist_ok=True, parents=True)
    (root / "tables").mkdir(exist_ok=True, parents=True)

    def f_exp(x): return float(np.exp(x))
    def f_sinpix(x): return float(np.sin(np.pi * x))

    tests = [("exp", f_exp, -1.0, 1.0),
             ("sin_pi_x", f_sinpix, -1.0, 1.0)]
    ns = [4, 8, 16, 32]

    records = []
    for name, f, a, b in tests:
        ref = float(mp.quad(lambda t: f(float(t)), [a, b]))
        for n in ns:
            for rule in ["cheb_gauss", "cheb_type2", "legendre"]:
                approx = float(integrate_on_ab(f, a, b, rule, n))
                rel_err = abs(approx - ref) / max(1.0, abs(ref))
                records.append({

```

```

        "test": name, "a": a, "b": b, "n": n, "rule": rule,
        "approx": approx, "reference": ref, "rel_error": rel_err
    })

df = pd.DataFrame.from_records(records)
summary = (df.groupby(["rule", "n"])["rel_error"]
           .mean().reset_index()
           .sort_values(["n", "rule"]))

with open(root / "tables" / "cheb_vs_legendre_errors.tex", "w") as fh:
    fh.write(summary.rename(columns={
        "rule": "Regra",
        "n": "$n$",
        "rel_error": "Erro relativo médio"
    }).to_latex(index=False, float_format="%.3e"))

# Figures per test
for test_name in df["test"].unique():
    sub = df[df["test"] == test_name]
    pivoted = sub.pivot(index="n", columns="rule",
        ↪ values="rel_error").sort_index()
    plt.figure()
    for rule in ["cheb_gauss", "cheb_type2", "legendre"]:
        if rule in pivoted.columns:
            plt.loglog(pivoted.index.values, pivoted[rule].values, marker="o",
                ↪ label=rule)
    plt.xlabel("n (nós)"); plt.ylabel("Erro relativo"); plt.title(f"Convergência -
        ↪ {test_name}")
    plt.legend(); plt.savefig(root / "figures" /
        ↪ f"cheb_gauss_vs_legendre_{test_name}.png", dpi=200, bbox_inches="tight");
    ↪ plt.close()

avg = (df.groupby(["rule", "n"])["rel_error"]
       .mean().reset_index()
       .pivot(index="n", columns="rule", values="rel_error")
       .sort_index())

plt.figure()
for rule in ["cheb_gauss", "cheb_type2", "legendre"]:
    if rule in avg.columns:
        plt.loglog(avg.index.values, avg[rule].values, marker="o", label=rule)
plt.xlabel("n (nós)"); plt.ylabel("Erro relativo médio (2 funções)")
plt.title("Convergência média: Chebyshev I, Chebyshev II e Gauss-Legendre")
plt.legend(); plt.savefig(root / "figures" / "cheb_gauss_vs_legendre.png",
    ↪ dpi=220, bbox_inches="tight"); plt.close()

```

### 5.3 Observações finais

- Todos os scripts foram testados em ambiente Linux Ubuntu 22.04 com Python 3.11.8.
- Os gráficos foram gerados automaticamente em formato PNG e salvos na pasta /figures.
- As tabelas de erros e matrizes numéricas foram exportadas em formato  $\text{\LaTeX}$  (.tex) e CSV.

- O uso de ambientes virtuais é fortemente recomendado para garantir reprodutibilidade dos experimentos.

## 6 Declaração de uso de LLM

Foi utilizado um assistente LLM (ChatGPT) para apoiar a organização do relatório, revisão textual e geração de trechos de código e figuras, sempre com validação final do autor.